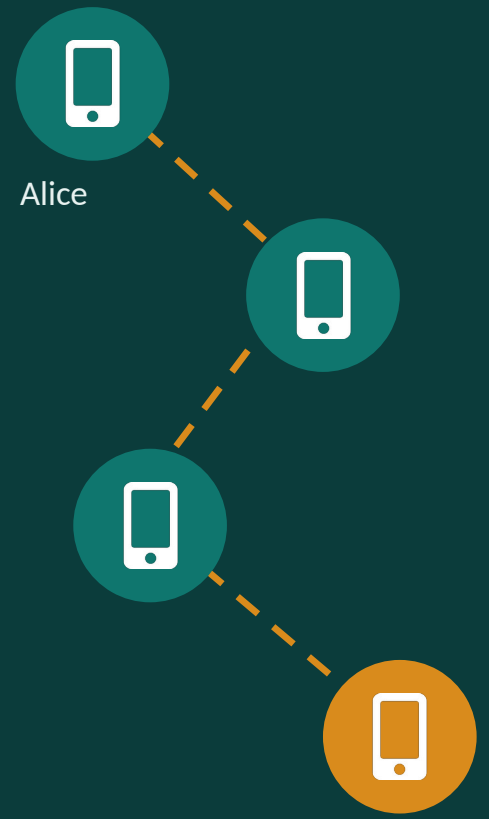
MeshPay

Paying through UPI when there is no internet

The problems, our ideas, and how we built them, in simple words

Java 17 | Spring Boot 3.5 | H2 database | Next.js dashboard



What is MeshPay?

1. You are in a basement. No internet.
2. You still tap Pay ₹500 to a friend.
3. Your phone locks the payment and hands it to nearby phones.
4. One phone walks outside, gets signal, and delivers it to the server.
5. The server unlocks it, checks it, and moves the money once.



The phone mesh is simulated in software, so the whole flow runs on one laptop.

Spring Boot backend | **Mesh simulator** | **H2 database** | **Dashboard** | **4 automated tests**

Four problems we had to solve



PROBLEM 1

How does a payment travel with no internet?

Gossip / flooding



PROBLEM 2

How do we stop strangers reading or editing it?

Encryption



PROBLEM 3

What if the same payment arrives many times?

Idempotency



PROBLEM 4

How do we move the money correctly, once?

Safe transaction + checks

Orange = the problem. Teal = our mechanism.

Problem 1: There is no internet

THE PROBLEM

Alice taps Pay in a basement. Her phone cannot reach the server.

WHY IT MATTERS

With no internet the payment is stuck on her phone, and Bob is never paid.

OUR IDEA

Let nearby phones carry it, like passing a note hand to hand, until someone gets internet.

IN OUR PROJECT

Gossip / flooding: every phone copies the packet to all phones near it. A hop limit (TTL = 5) stops it spreading forever.



A real-life picture

Like passing a sealed letter through a crowd until someone walks out and drops it into a post box.

Problem 2: Strangers carry our payment

THE PROBLEM

The phones that carry the payment belong to strangers.

WHY IT MATTERS

A stranger could read the amount and names, or quietly change ₹500 into ₹5000.

OUR IDEA

Seal the payment in a lock only the server can open. Any change breaks the seal.

IN OUR PROJECT

Hybrid encryption (RSA + AES-GCM). A tampered packet simply fails to open.



A real-life picture

A padlock anyone can snap shut, but only the bank holds the key to open it. Tamper with the box and the seal shows it.

Problem 3: The same payment arrives many times

THE PROBLEM

Many phones hold copies of the same payment. Several may reach internet at the same moment.

WHY IT MATTERS

Without a guard, Alice could be charged ₹100 three times instead of once.

OUR IDEA

Give each payment a fingerprint. The first copy wins; later copies are turned away.

IN OUR PROJECT

Idempotency: the packet's SHA-256 fingerprint is claimed in one atomic step (putIfAbsent).



A real-life picture

A cinema ticket has one serial number. Once it is scanned, a photocopy of it is refused at the door.

Problem 4: Money must move correctly

THE PROBLEM

Paying means taking money from Alice and giving it to Bob.

WHY IT MATTERS

A crash halfway loses money. An old copied packet could be replayed. Two updates could overwrite each other.

OUR IDEA

Do both steps as one unit, check the packet is recent, and use a version number to catch clashes.

IN OUR PROJECT

@Transactional settlement, 24-hour freshness check, @Version on Account, balance check.



A real-life picture

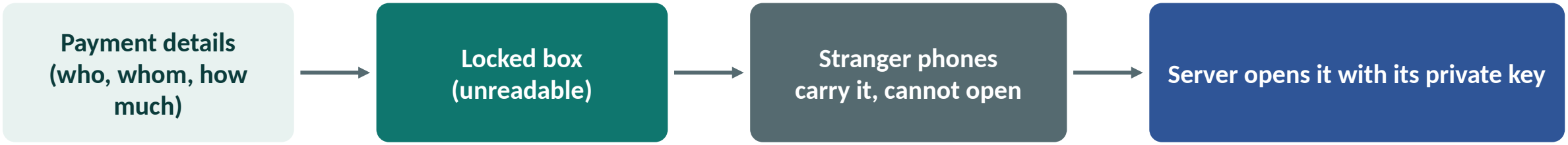
Moving cash between two lockers: both moves happen, or neither. A cheque dated last month? Rejected.

Our approach: one tool for each problem

PROBLEM	OUR MECHANISM	WHERE IN THE CODE
 No internet	→ Gossip / flooding with TTL	`MeshSimulatorService`, `VirtualDevice`
 Strangers carry it	→ Hybrid encryption	`HybridCryptoService`, `ServerKeyHolder`
 Duplicates	→ Idempotency (fingerprint claim)	`IdempotencyService`
 Money must move right	→ Transaction + freshness check + version number	`SettlementService`, `BridgeIngestionService`, `Account`

Story: problem → why it hurts → our idea → mechanism → code.

Cryptography: locking the payment



Encryption

Scrambling data so only the right person can read it.

Public and private key

Public key = a padlock anyone can close. Private key = the only key that opens it. It stays on the server.

Why two locks?

The padlock (RSA) is slow and can only hold something small. So we lock a small one-time key with it, and use a fast lock (AES) for the real message.

A tampered packet does not open. It is rejected, never settled.

Cryptography in our project

What happens to one payment

1. Build the payment: sender, receiver, amount, hashed PIN, random nonce, time.
2. Make a brand-new secret key, used only for this packet.
3. Lock the payment with that secret key (AES-GCM).
4. Lock the secret key with the server's public key (RSA-OAEP).
5. Server reverses it: opens the small lock, then the big lock.

Where: `HybridCryptoService.encrypt() / decrypt()`

Where: `ServerKeyHolder (makes the RSA key pair at start-up)`

What the packet carries

1. Secret key, locked with RSA (256 bytes)

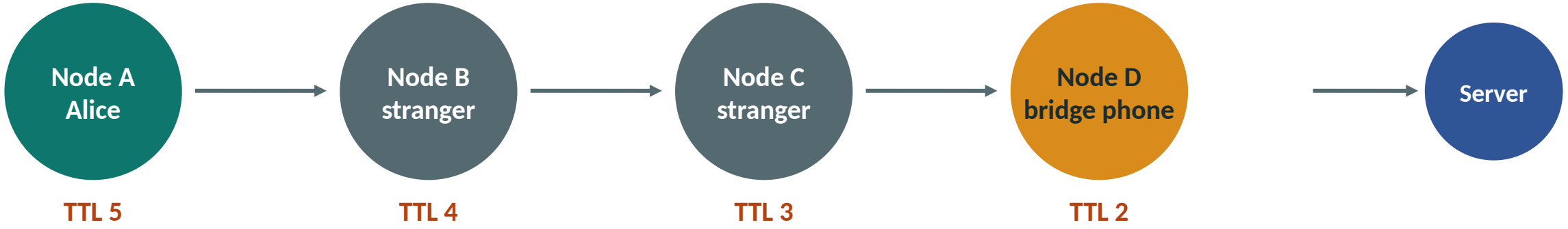
2. Random starter number, called IV (12 bytes)

3. Locked payment + tamper seal (data + 16 bytes)

```
// test: tamperedCiphertextIsRejected
flip ONE character  →  INVALID
reason: decryption_failed
```

PIN is hashed and recorded only; it is not verified in this demo.

Flooding / gossip: spreading the word



Each hop copies the packet and lowers its TTL by 1. The bridge phone then hands it to the server.

What is flooding?

Tell everyone you meet. Whoever can reach the internet first delivers it.

Why gossip?

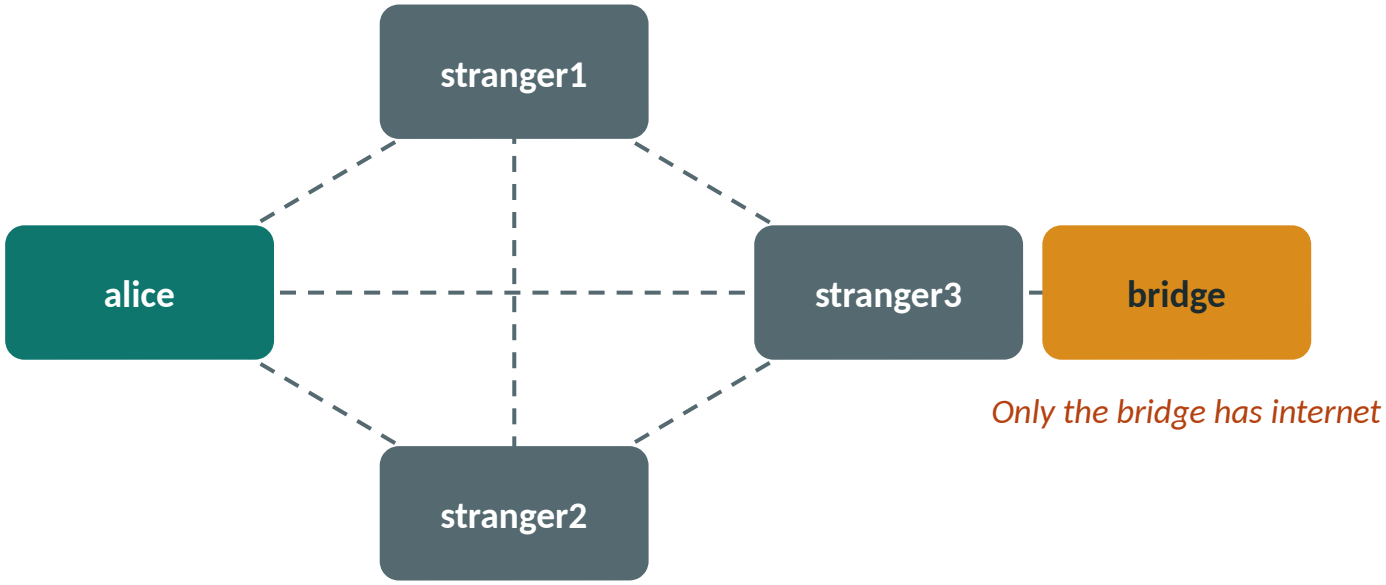
Like a rumour: every person tells everyone they meet. No map of the network needed.

What is TTL?

A counter on each copy. It drops by 1 at every hop and the copy stops at 0.

Cost: lots of copies. That is why we need idempotency later.

Gossip in our project



Only the bridge has internet

5 simulated phones. Every phone is in range of every other.

Where: `MeshSimulatorService.gossipOnce()`

Where: `VirtualDevice.hold()` ignores a packet it already has

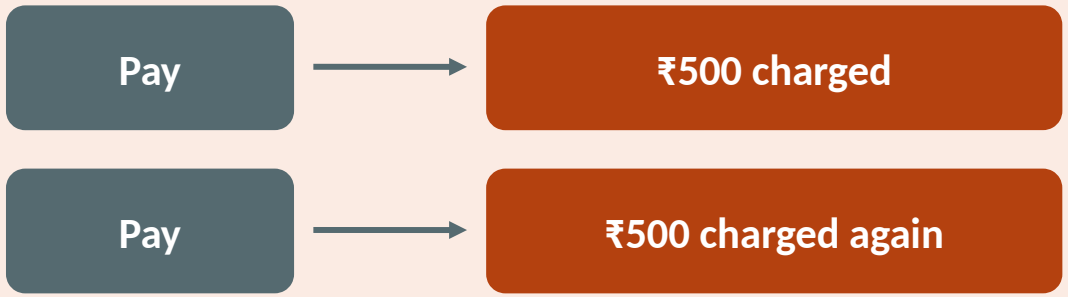
One gossip round

1. Take a snapshot of what every phone holds.
2. Each phone gives each packet to every phone that does not have it.
3. TTL goes down by 1 on every copy.
4. Packets at TTL 0 are not passed on.
5. Then 'flush': the bridge phone uploads everything to /api/bridge/ingest.

Real Bluetooth is not built: this is a software simulation.

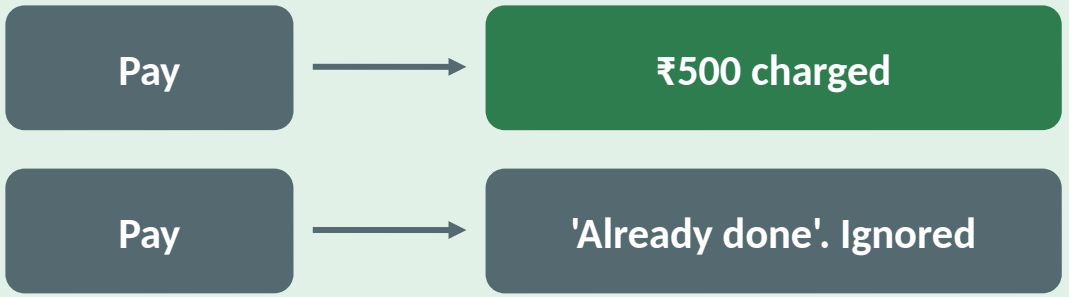
Idempotency: pay twice, charged once

WITHOUT idempotency



Alice loses ₹1000 for one payment.

WITH idempotency

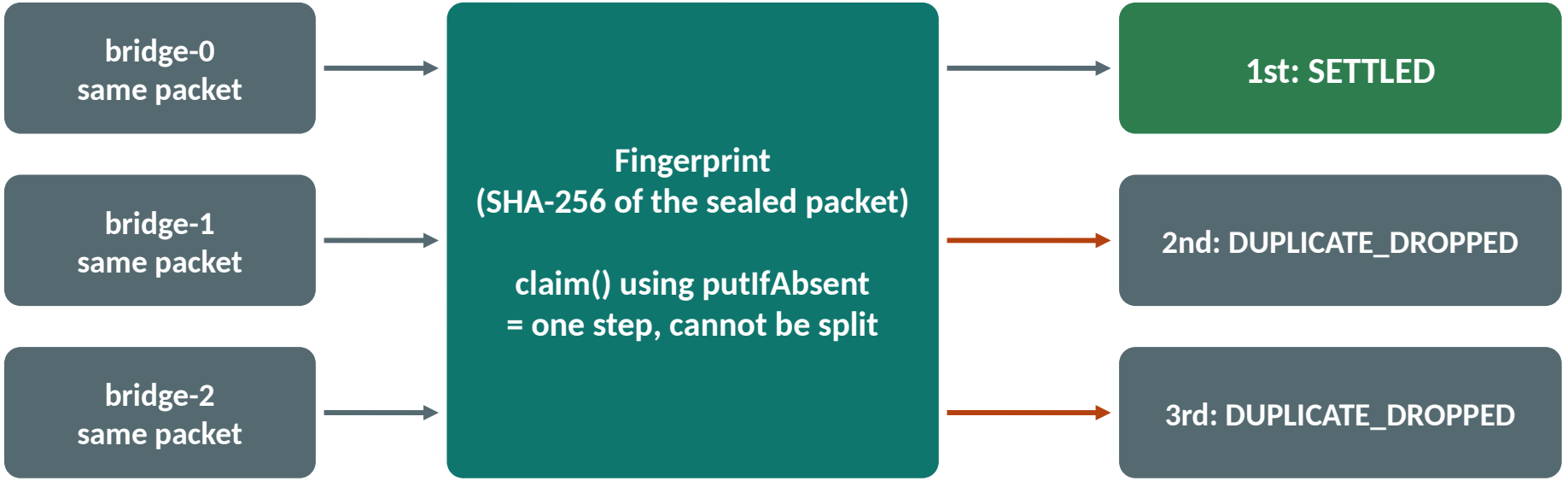


Alice pays ₹500 once. The copy is dropped.

Idempotent means: doing it again gives the same result as doing it once. Like a lift button: press it five times, the lift still comes once.

In our project the 'double click' is the same packet delivered by several phones.

Idempotency in our project



Why hash the sealed packet, not the packet ID?

Strangers can rewrite the ID. They cannot forge sealed contents.

```
return seen.putIfAbsent(packetHash, Instant.now()) == null;
// true = first caller, false = duplicate
```

Test: 3 threads, same packet → 1 settled, 2 dropped. Alice debited once.

Where: `IdempotencyService.claim()`

Where: `BridgeIngestionService.ingest()`

Backup: a unique index on packetHash in the database. A background task cleans old fingerprints every 60 seconds.

Key features (1 of 2)

Encrypted packets

What: Payment details are locked before they leave Alice's phone.
Why: Strangers carry the packet.
Benefit: Nobody on the way can read or change it.

Where: `HybridCryptoService`

Mesh simulator

What: Five virtual phones that pass packets around.
Why: Show the whole flow on one laptop, no Bluetooth needed.
Benefit: Anyone can try it and see it.

Where: `MeshSimulatorService, VirtualDevice`

Duplicate protection

What: A fingerprint check where the first copy wins.
Why: Many phones deliver the same payment.
Benefit: Money moves only once.

Where: `IdempotencyService`

Freshness check

What: Each payment carries the time it was signed.
Why: Stop old copied packets being replayed.
Benefit: Over 24 h old or over 5 min in the future: rejected.

Where: `BridgeIngestionService`

Key features (2 of 2)

Safe settlement

What: Debit, credit and ledger record done as one unit.

Why: A crash halfway must not lose money.

Benefit: All happens, or nothing does.

Where: `SettlementService`

Dashboard + REST API

What: A web page and 9 API endpoints (send, gossip, flush, balances, ledger…).

Why: See what is happening live.

Benefit: Easy to demo and test.

Where: `ApiController, dashboard.html, frontend`

Clear error reasons

What: Every failure comes back with a reason such as decryption_failed.

Why: Easy to debug, and the server never crashes.

Benefit: Users get a clean answer, not a stack trace.

Where: `BridgeIngestionService, GlobalExceptionHandler`

Tests + CI

What: 4 automated tests, including a 3-thread race test; GitHub Actions on Linux and macOS.

Why: Prove the claims, not just say them.

Benefit: Confidence that it works.

Where: `IdempotencyConcurrencyTest, ci.yml`

OOP (1/3): Classes and objects

Class = blueprint. Object = a real thing built from the blueprint. Account is the class; alice@demo with ₹5000 is an object.

Class	What it represents	Example object
`Account`	A bank account	alice@demo, ₹5000
`Transaction`	One ledger entry	₹100 alice → bob, SETTLED
`MeshPacket`	The envelope passed between phones	packetId, ttl = 5, locked data
`PaymentInstruction`	Payment details inside the lock	alice → bob, ₹500, nonce
`VirtualDevice`	One simulated phone	phone-bridge (has internet)
`HybridCryptoService`	The locksmith (lock / unlock)	one shared service object
`Idempotency + SettlementService`	The doorkeeper and the cashier	one shared object each

Model classes hold data. Service classes do work. Each class has one clear job.

OOP (2/3): Encapsulation and abstraction

Encapsulation

Keep the data private. Allow only safe doors to reach it.

- VirtualDevice keeps its packets in a private field. Outside code can only call hold(), holds() or packetCount().
- ServerKeyHolder keeps the private key inside. Only the crypto service asks for it.
- Account and Transaction fields are private. Getters and setters are generated by Lombok.

```
private final Map<String, MeshPacket>
        heldPackets = new ConcurrentHashMap<>();
```

Abstraction

Hide the hard part behind a simple call.

`crypto.decrypt(text)` → hides RSA, AES, and byte slicing

`accountRepo.findAll()` → hides the SQL

`bridge.ingest(packet…)` → hides the whole 5-step pipeline

The controller stays short because the services hide the details.

OOP (3/3): Inheritance and polymorphism

Inheritance

A child gets the parent's abilities for free.

```
interface AccountRepository
   extends JpaRepository<Account, String>
interface TransactionRepository
   extends JpaRepository<Transaction, Long>
```

We get save(), findById(), findAll() and more without writing them.

Also: WebConfig implements WebMvcConfigurer.

Polymorphism

One name, different real objects behind it.

- A Map variable holds a ConcurrentHashMap object (VirtualDevice, IdempotencyService).
- WebConfig overrides addCorsMappings(). Spring calls our version.
- We write only the repository interface. Spring builds the real object at run time.

Viva-ready: we do not have our own parent-child class hierarchy. Our design prefers combining objects. We use inheritance through the repositories.

File handling: what our project does with files

What our Java code does

It does not read or write data files. Payments, balances and the ledger are stored in the database, not in files.

Files the project does use

`application.properties` settings read at start-up
`dashboard.html` page served at /
`pom.xml` build and libraries
`ci.yml` automatic build and test

What is in application.properties

```
server.port=${PORT:8080}
upi.mesh.packet-max-age-seconds=86400
upi.mesh.idempotency-ttl-seconds=86400
spring.datasource.url=jdbc:h2:mem:upimesh
```

If a setting is missing

- The code carries safe defaults, for example 86400 seconds (24 h).
- The port falls back to 8080 when PORT is not set.

If asked: we could export the ledger to CSV using try-with-resources and catch IOException. Not built yet.

Files hold settings and pages. The database holds the money.

Exception handling: not crashing when things go wrong

Exception = something unexpected that interrupts the program. try / catch lets us deal with it instead of crashing.

What goes wrong	What Java does	What the user gets
Packet tampered or junk	decrypt() throws, we catch it	INVALID: decryption_failed
Packet too old or from the future	Time check fails	INVALID: stale_packet / future_dated
Not enough balance	No error: we record it	INVALID: insufficient_balance
Unknown account, amount ≤ 0	settle() throws IllegalArgumentException	INVALID: internal_error (caught)
Blank or bad request fields	@Valid check fails	HTTP 400: validation_error
Anything else unexpected	GlobalExceptionHandler	HTTP 500: a safe, generic message

```
try {
  instr = crypto.decrypt(ct);
} catch (Exception e) {
  return INVALID;
  // decryption_failed
}
```

@Transactional also undoes a half-finished settlement if an error is thrown.

Where: `BridgeIngestionService (try/catch), GlobalExceptionHandler`

We use try / catch. No finally block was needed: Spring closes connections for us.

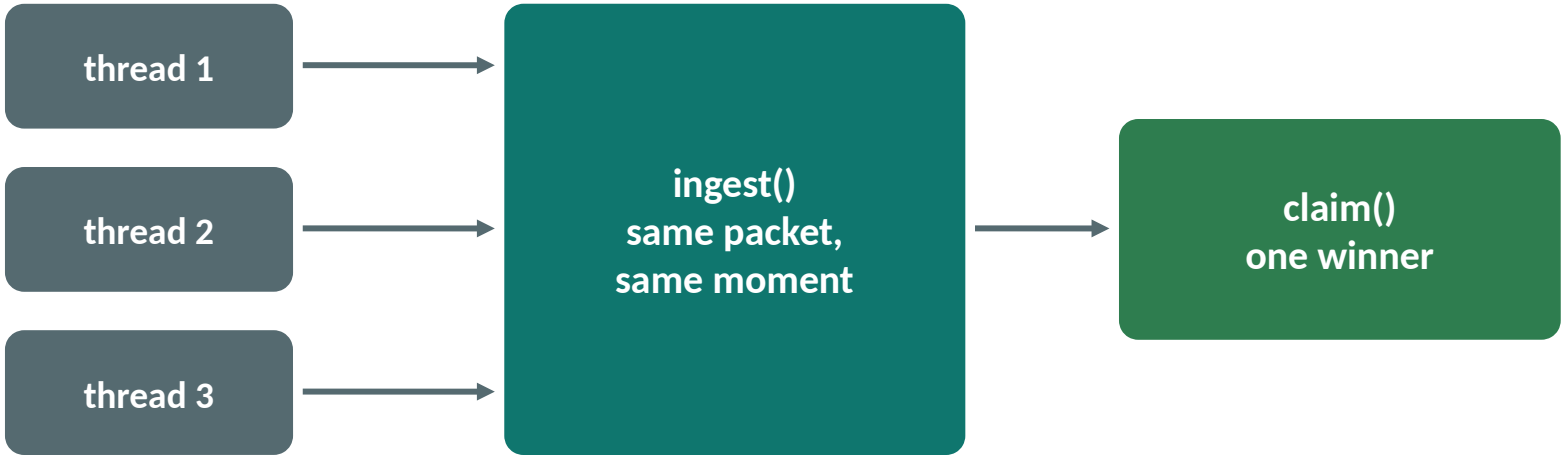
Collections we actually use

Collection	In simple words	What we store and where
`ConcurrentHashMap`	A phone book that many threads can use safely	fingerprint → time (IdempotencyService); packetId → packet (VirtualDevice); id → phone (MeshSimulatorService)
`ArrayList`	A flexible list: add and remove items	list of devices, bridge uploads, ingest results
`HashMap`	A phone book, no safety for threads	snapshot of what each phone holds at the start of a gossip round
`LinkedHashMap`	A phone book that remembers order	phone → packet count, so the dashboard shows phones in order
`Collections.synchronizedList`	A list safe for many threads	collects results from the parallel upload
`Map.of / List.of / record`	Fixed bundles that cannot be changed	JSON answers the API sends back

Why Map? We always look things up by a name: a fingerprint or a packet ID.

Multithreading: many jobs at once

One person doing five jobs one by one is slow. Five people doing them at the same time is fast. Each 'person' is a thread.



How we keep it safe

- ConcurrentHashMap
- putIfAbsent: one atomic step
- synchronizedList for results
- @Version on Account

`parallelStream()`

The flush uploads from all bridge phones at the same time (ApiController).

`Web server threads`

Each incoming request runs on its own thread, so real bridges can hit /ingest together.

`@Scheduled`

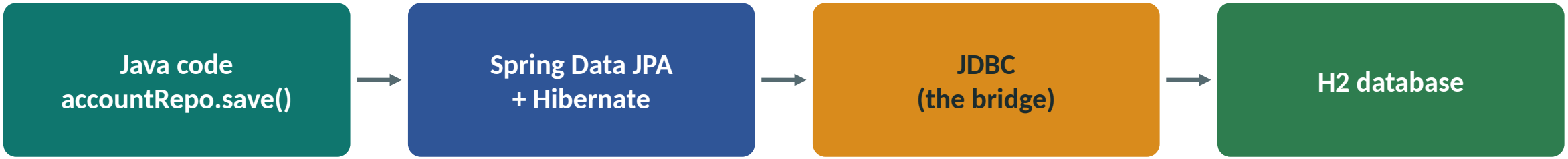
A background thread removes old fingerprints every 60 seconds.

`The test`

ExecutorService + CountDownLatch start 3 threads together.

JDBC: the bridge to the database

We do not write JDBC code by hand. Spring Data JPA does it for us, and it uses JDBC underneath.



Job	What we write in Java	What happens underneath
Connect	`spring.datasource.url=jdbc:h2:mem:upimesh`	Spring opens and manages connections
Insert	`transactions.save(tx)`	INSERT INTO transactions…
Read one / all	`accounts.findById(vpa), findAll()`	SELECT … FROM accounts
Latest 20	`findTop20ByOrderByIdDesc()`	SELECT … ORDER BY id DESC, 20 rows
Update	`accounts.save(sender)`	UPDATE … WHERE id = ? AND version = ?

Closing connections and handling SQL errors is also done by Spring. We never delete rows: the ledger is permanent.

Database integration

Database used: H2 (in memory). MySQL is not used. For a real system the README suggests PostgreSQL or MySQL.

`accounts`

`vpa` main ID, like alice@demo
`holderName` who owns it
`balance` money, exact decimal
`version` catches two updates at once

4 demo accounts: alice ₹5000, bob ₹1000, carol ₹2500, dave ₹500

`transactions` (one row per payment)

`id, packetHash` hash is UNIQUE (the fingerprint)
`senderVpa, receiverVpa, amount` who paid whom
`signedAt, settledAt` when signed / when settled
`bridgeNodeId, hopCount` how it arrived
`status` SETTLED or REJECTED



Data is lost when the server stops (create-drop). Switching to MySQL is mostly a configuration change.

Servlets: how a request reaches our code

We do not write Servlet classes. Spring Boot's built-in servlet receives each request and hands it to our controller.

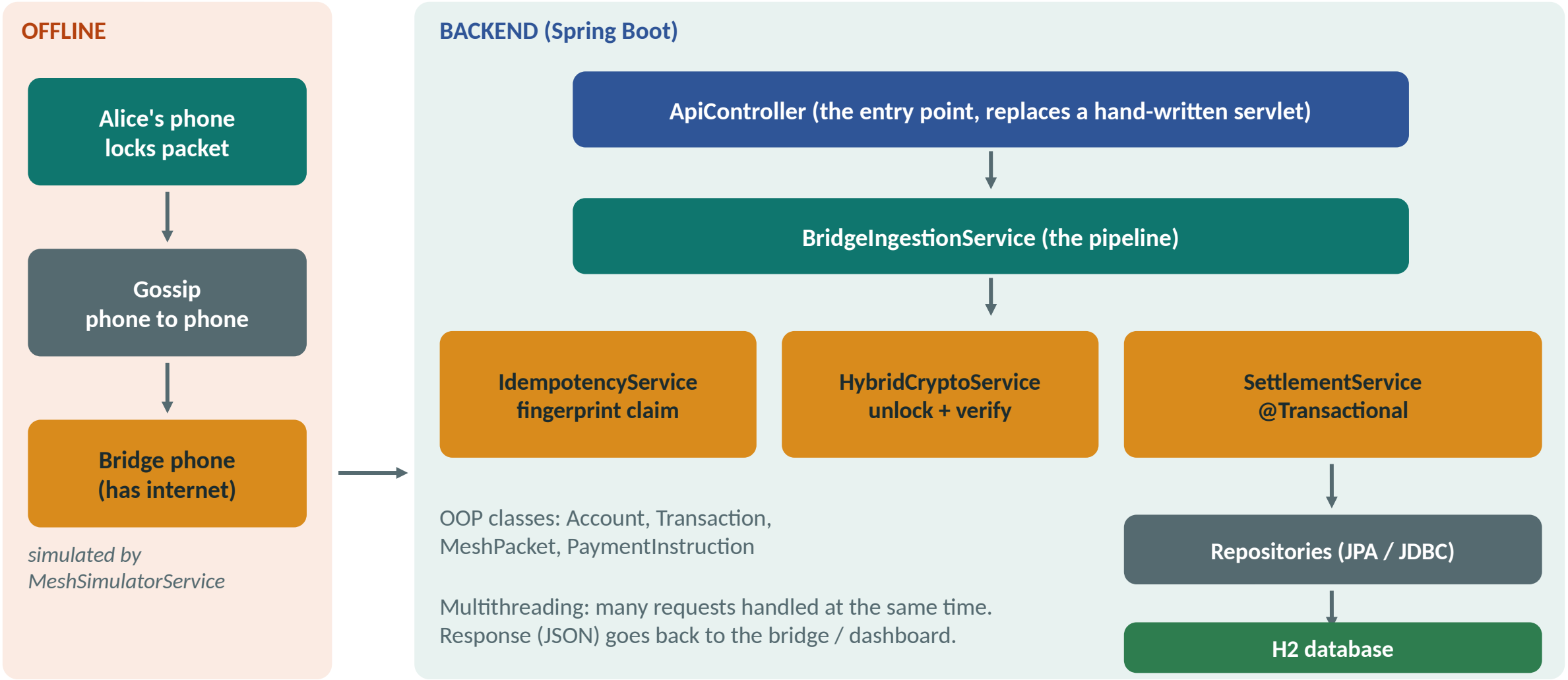


Request goes in → Response (JSON) comes back ←

Plain servlet world	What we write instead
`doPost(request, response)`	`@PostMapping("/bridge/ingest")`
`doGet(request, response)`	`@GetMapping("/accounts")`
`request.getHeader("X-Bridge-Node-Id")`	`@RequestHeader("X-Bridge-Node-Id")`
`read and parse the request body`	`@RequestBody MeshPacket packet`
`write the response yourself`	`return ResponseEntity.ok(result)`

Our controller has 9 API endpoints plus one that serves the dashboard page at /.

Complete architecture



One payment, start to finish

1 Alice taps Pay ₹500

Phone builds and locks the packet.

2 Phones gossip

Copies spread, TTL 5 → 4 → 3.

3 Bridge gets signal

It POSTs to /api/bridge/ingest.

4 Controller receives

Reads node ID and hop count.

5 Fingerprint claimed

First copy wins. Later copies are dropped.

6 Server unlocks packet

Tampered? Rejected right here.

7 Freshness check

Within 24 hours? Not future-dated?

8 Money moves

Alice −500, Bob +500, ledger row saved.

9 Answer returns

SETTLED. Dashboard shows new balances.

A copy at step 5 is dropped. A tampered packet stops at step 6. Money moves only at step 8.

What we achieved

Four problems solved

No internet, strangers, duplicates, correct money.

Secure

Locked packets, tamper detection, 24-hour freshness window.

Reliable

Exactly-once settlement, all-or-nothing transactions.

Database integrated

JPA + H2 with accounts and a permanent ledger.

Concurrent

Parallel uploads, thread-safe collections.

Clean design + errors

Layered classes; clear reasons; a global safety net.

Plus: 4 automated tests, including a 3-thread race test, and automatic builds on GitHub Actions.

Honest limits and next steps

Limits today

- No sender signature: anyone with the public key can build a packet.
- An offline payment is an IOU. It can be rejected later if funds are missing.
- The phone mesh is simulated, not real Bluetooth.
- H2 and the server keys reset on restart.
- The fingerprint list lives in one server's memory.
- The ingest endpoint has no login or rate limit.

Next steps

- Sender signatures to prove who sent it.
- Redis for the shared fingerprint check.
- MySQL or PostgreSQL for permanent data.
- Keys held in a secure vault, not in memory.
- A real phone app using Bluetooth.
- Do not lose a payment if settlement fails after the claim.

Conclusion

What it does	Delivers an offline payment through strangers' phones and settles it once.
Problems solved	No internet, untrusted carriers, duplicates, and money correctness.
How	Gossip carries it, encryption protects it, idempotency de-duplicates it, a transaction settles it.
Together	Spring Boot controllers and services, JPA with a database, and thread-safe collections.
Why it is useful	It shows how to build safe payments on unreliable networks.